WILLKOMMEN

Python Tutorium: Strukturen, Logik & Methoden

Wie findet ihr die Vorlesung?
Habt ihr den Stoff bereits nachgeholt?

1. Typkonvertierung & Zahlen

Zahlen-Basen

🧮 **bin & hex:** Umrechnung in Dezimal.

√× **Mathematisch:** Darstellung der Umrechnung.

Konvertierungs-Methoden

`int('3.7')`: Löst Fehler aus (funktioniert nicht).

`round()`: Rundet Nachkommastelle je nach Größe.

`str(13.70)`: Speichert keine überflüssigen Nullen.

Zeichen & Unicode

`ord()`: Codepoint als Ganzzahl.

`chr()`: Codepoint zurück zu String.

`ord('ab')`: Fehler (nimmt nur ein Zeichen).

2. Boolean & Logik

Wahrheitswerte

Arithmetik: True=1, False=0 (Berechnungen möglich).

Falsy-Werte: Leeres wird False.

```
0, [], (), set(), None
```

== : Werte der Variablen gleich.

is : Variablen referenzieren selbes Objekt.

Logische Verknüpfungen

x and y

x falsch: Ausgabe x (Aussage falsch).

x wahr: Ausgabe y (hängt von y ab).

x or y

x falsch: Ausgabe y (hängt von y ab).

x wahr: Ausgabe x (Aussage immer wahr).

3. Listen: Grundwissen & Slicing

- Index startet bei 0.
- Index -1 ist letztes Element (von hinten).
- Index darf (Länge - 1) nicht überschreiten.
- **Verschachtelt:** Liste in Liste möglich.
- **Aliasing:**
- **OOP:** Liste ist sowohl Objekt als auch Klasse (besitzt Methoden).
- **Speicher:** Array (Kästchen) vs. Dictionary (Key/Item).

Slicing Syntax

```
liste[start:ende:schritt]
```

- → start = inklusiv, ende = exklusiv.
- → Schritt default 1
- → Ohne ende: Iteriert bis zum Schluss.
- → Ohne Argumente: Erstellt eine Kopie.

4. Listen: Operationen & Basis-Methoden

`len()`

Länge der Liste (Anzahl der Elemente).

`list()`

Erstellung eines Listen-Objekts.

`dir()`

Alle Attribute & Methoden der Klasse.

Addition: `liste + liste`

Erzeugt neues Objekt. Originale bleiben unverändert.

Multiplikation: `liste * zahl`

Erzeugt neues Objekt. Vergleichbar mit String.

5. Listen: Bearbeitungsmethoden

Methode	Effekt / Funktion
`.append() / .extend()`	Fügt ein Element / alle Elemente einer anderen Liste hinzu.
`.insert()`	Fügt ein Element an der n-ten Stelle hinzu.
`.pop()`	Zeigt und entfernt das n-te Element (default: letztes).
`.remove()`	Entfernt ein bestimmtes Element (das erste Vorkommen).
`.count() / .index()`	Anzahl der Elemente / Index des ersten Vorkommens.
`.sort() vs sort(liste)`	Aufsteigend. `.sort()` ändert Original, `sort()` ändert es nicht.
`.reverse() vs reverse(liste)`	Absteigend. Gleiches Prinzip bezüglich Änderung des Originals.
`.clear()`	Löscht alle Elemente in der Liste.

6. Unveränderbare Sequenzen: String & Tupel

String-Methoden

Iterierbar wie Liste, aber **nicht veränderbar**.

`sorted`: Zeichen zerlegen, aufsteigend.

`.upper() / .lower()`: Groß / klein geschrieben.

`.isalpha() / .isdigit()`: Prüft Buchstaben / Zahlen.

`.startswith() / .endswith()`: Prüft Muster-Ränder.

`.replace(x,y)`: x durch y ersetzen.

`.rjust() / .center()`: Rechtsbündig / zentriert in einem Feld einer angegebenen Länge ausrichten.

Tupel Grundlagen

Iterierbar, unveränderbar

- `(1)` ist **kein** Tupel, nur eine Ziffer in Klammern.
- Iterierbar durch Slicing (wie Liste).
- `list()` wandelt Tupel in Liste um.

Methoden:

`.count()` `.index()`

7. Set (Mengen)

Exakt wie mathematische Mengen. **Keine Index-Reihenfolge. Keine Duplikate.**

`.add()`

Element hinzufügen (ähnlich .append).

`- (Minus)`

Differenz: Elemente aus einer andern Menge ausräumen.

`| / .union()`

Vereinigung: Beide Sets kombinieren.

`& / .intersection()`

Schnittmenge: Gemeinsame Elemente aus zwei Mengen

`.issubset()`

Teilmenge: Prüft enthaltene Elemente.

8. Kontrollstrukturen & Schleifen

If-Bedingungen

- `If & else` oder `If & elif & else`.
- **Einrückung:** Garantiert Logik und Ausführung.
- **Einzeiler:** Kompakte Darstellung (im Grund gleich).

Zusatz

- `divmod(x,y)` liefert Quotienten & Rest (`//` und `%`).
- `range(x)`: Zahlen von 0 bis x-1 (x ist ausgeschlossen).

Schleifen (For & While)

- **For:** Liefert iterierbare Elemente nacheinander.
- **While:** Codeblock läuft unendlich, falls die Bedingung wahr ist.

Schleifen-Steuerung:

`break:` Beendet Schleife sofort.

`continue:` Springt zur nächsten Iteration.

`return:` Gibt das Ergebnis zurück und macht noch break.

9. Funktionen & Rekursion

Syntax & Verhalten

`def name():` Erstellung.

`name()` vs **`name`**: Aufruf vs Objekt-Referenz.

`pass`: Block überspringen.

Argumente: Strikte Anzahl, dürfen nicht fehlen oder überschreiten

`return`: Liefert Ergebnis, bricht Funktion ab.

Ohne `return` liefert Python unsichtbar `None`.

Scope: Lokal & Global

- **Lokal:** Existiert nur im Funktions-Block.
- **Global:** Ohne Einrückung, für ganzen Code gültig.
- Funktion liest Global, aber Global liest Lokal nicht.
- Befehl `global` ändert Zuweisung lokal.

Rekursion

Funktion ruft sich selbst auf, bis Abbruchbedingung wahr wird.

10. Kommentare, Docstrings & f-Strings

Dokumentation & Zeichen

- **Hash (#):** Einzeiliger Kommentar (ignoriert).
- **Docstring (f'''...'''):** Mehrzeilig. es ist unter Funktion via `.__doc__` aufrufbar.

Escape-Sequenzen

`\r` : Wagenrücklauf (Cursor zum Zeilenanfang).

`\b` : Backspace (ein Zeichen nach links).

`\a` : Erzeugt echten Ton.

Formatierte Strings (f-Strings)

Variablen in `{}`

Doppelte `{{}}` sind normale Klammern

`f'{zahl:.2f}'`	Runden wie round(), zwei Nachkommastellen behalten.
`f'{str:zahl}'`	Linksbündig auffüllen.
`f'{str:`	Rechtsbündig (.rjust).
`f'{str:^zahl}'`	Zentriert (.center).
`f'{int:,}'`	Tausendertrennzeichen.
`f'{var=}'`	Gibt `var = Wert` aus.
`f'{eine_zahl:Mindestlänge Füllzeichen}'`	Spezifisches Auffüllen.

11. Die Kommandozeile (CLI)

Command Line Interface

Steuerung des Systems ganz ohne grafische Oberfläche (GUI).

- **Shells:** Nutzung von bash oder zsh.

Vergleich zu Windows

- Kommandozeile existiert auch bei Windows und Mac
- **Beispiele:** `cmd / powershell; terminal`
- **Aber:** bei Windows völlig andere Befehle; bei Mac ähnlich

12. Befehlsstruktur & Fernzugriff

Grundstruktur der Shell

```
Befehl [Leerzeichen] [Optionen] [Leerzeichen] Argumente
```

Fernsteuerung per SSH

```
ssh schaefeb@remote.cip.ifi.lmu.de
```

ssh: Der Befehl. **schaefeb:** Username. **@:** Trennzeichen. **remote...de:** Hostname

13. Navigation im Dateisystem

Dateisystem ist als Baum organisiert (Anzeige z.B. durch `tree -L 1`).

Der Prompt (Wer & Wo)

```
schaefeb@mittenwald.cip.ifi.lmu.de:~ $
```

schaefeb: Loginname.

mittenwald...: Rechnername / Host.

~ (Tilde): Befindet sich im Homeverzeichnis.

Wichtige Befehle

`ls` `cd` `mkdir` `rmdir` `pwd` `cp` `mv` `rm`

Hilfe aufrufen:

`man ls` oder `ls --help`

14. Shortcuts & Wichtige Regeln

Tabulator-Taste

Für die Autovervollständigung von Befehlen und Pfaden.

History (Pfeil & Suche)

Pfeil-hoch: Vorherige Befehle.
Ctrl + r: Suche im Verlauf.

Case Sensitive

Dateinamen unterscheiden streng nach Groß- und Kleinschreibung!